Guía: desplegar una app Angular + Express en AWS

Oct 20, 2026 · @Ricardo

Objetivo

Esta guía describe cómo publicar una aplicación completa en internet: el **backend** (Node + Express) en AWS **ECS Express Mode** y el **frontend** (Angular) en AWS **Amplify**.

Requisitos previos:

- Una cuenta de AWS
- Docker Desktop instalado
- AWS CLI instalado y configurado (`aws configure`)
- El código de ambos proyectos en GitHub

Parte 1: El backend

Paso 1: Preparar Express

El servidor debe cumplir tres condiciones: escuchar en el puerto asignado, tener una ruta `/health` y permitir CORS. El paquete CORS se instala con `npm install cors`.

```
const express = require('express');
const cors = require('cors');
const app = express();

app.use(cors({ origin: process.env.FRONTEND_URL || '*' }));

app.get('/health', (req, res) => res.send('ok'));

app.listen(process.env.PORT || 3000);
```

Paso 2: Crear el Dockerfile

En la raíz del backend se crea un archivo llamado `Dockerfile` (sin extensión):

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
EXPOSE 3000
CMD ["node", "index.js"]
```

También se crea un archivo `.dockerignore` con una sola línea: `node_modules`

Paso 3: Subir la imagen a ECR

ECR (Elastic Container Registry) es el servicio de AWS para almacenar imágenes Docker.

1. En la consola de AWS, abrir **ECR** y seleccionar **Create repository**. Asignar un nombre, por ejemplo `mi-backend`.
2. Entrar al repositorio y seleccionar **View push commands**. AWS muestra los 4 comandos exactos para la cuenta y la región correspondientes.
3. Ejecutarlos en orden en la terminal: login, build, tag y push.

Paso 4: Crear el servicio en ECS Express Mode

1. En la consola, abrir **ECS**.
2. Crear un servicio en **Express Mode**.
3. Completar los datos:
   - **Imagen:** la del repositorio ECR.
   - **Puerto del contenedor:** 3000.
   - **Health check path:** `/health`.
   - **Roles** (task execution role e infrastructure role): permitir que AWS los **cree automáticamente**.
4. Crear el servicio y esperar unos minutos.

Al terminar, AWS entrega una **URL HTTPS**. Al abrirla en el navegador con `/health` al final, la respuesta `ok` confirma que el backend funciona.

Parte 2: El frontend

Paso 5: Apuntar Angular al backend

En el archivo de environment de Angular se guarda la URL del backend:

```
export const environment = {
  production: true,
  apiUrl: 'https://LA-URL-QUE-ENTREGO-AWS'
};
```

Esa URL se usa en los servicios (`environment.apiUrl + '/ruta'`). Luego se hace commit y push a GitHub.

Paso 6: Publicar en Amplify

1. En la consola, abrir **AWS Amplify**, seleccionar **Create new app** y elegir **Host web app**.
2. Elegir **GitHub**, autorizar el acceso y seleccionar el repositorio y la rama.
3. Revisar la configuración de build. Lo importante es la carpeta de salida: `baseDirectory: dist/NOMBRE-DEL-PROYECTO/browser`
4. Guardar y desplegar.

Paso 7: Configurar el build del monorepo (solo si el frontend está en una subcarpeta)

Este paso aplica únicamente cuando el repositorio contiene las carpetas `frontend/` y `backend/` en la misma raíz. Si el proyecto Angular está en la raíz del repositorio, se omite.

En la raíz del repositorio (junto a `frontend/` y `backend/` , no dentro de `frontend/`) se crea un archivo llamado `amplify.yml` , con extensión `.yml` , con este contenido:

```
version: 1
applications:
  - appRoot: frontend
    frontend:
      phases:
        preBuild:
          commands:
            - npm ci
        build:
          commands:
            - npm run build
      artifacts:
        baseDirectory: dist/NOMBRE-DEL-PROYECTO/browser
        files:
          - '**/*'
      cache:
        paths:
          - node_modules/**/*
```

Puntos a tener en cuenta:

- Los comandos se ejecutan dentro de `frontend/`, y `baseDirectory` es relativo a esa carpeta. Por eso se escribe `dist/...` y no `frontend/dist/...`.
- En la consola de Amplify, el campo **Monorepo root directory** debe contener `frontend`. Si no se llenó al crear la app, se agrega en **Hosting → Environment variables** la variable `AMPLIFY_MONOREPO_APP_ROOT` con el valor `frontend`. Debe coincidir con el `appRoot` del archivo.
- Opcional: agregar la variable `AMPLIFY_DIFF_DEPLOY` con el valor `true`, para que Amplify solo construya cuando cambien archivos dentro de `frontend/`. Sin ella, cada push que modifique solo el backend también dispara un build.

Paso 8: Arreglar las rutas de Angular

Sin este paso, al recargar la página en una ruta como `/perfil` aparece un error 404.

1. En Amplify, ir a **Hosting → Rewrites and redirects**.
2. Agregar la siguiente regla:
   - **Source:** `</^[^.]+$|\.(?!(css|gif|ico|jpg|js|png|txt|svg|woff|woff2|ttf|map|json|webp)$)([^.]+$)/>`

- **Target:** `/index.html`
- **Type:** `200 (Rewrite)`

Parte 3: Conectar todo

Paso 9: Permitir el dominio de Amplify en CORS

Amplify asigna una URL como `https://main.xxxxx.amplifyapp.com`. El backend debe aceptarla:

1. Colocar esa URL en la variable de entorno `FRONTEND_URL` del servicio en ECS (o directamente en el código de CORS).
2. Si se modificó código, repetir el **Paso 3** (build, tag y push) y actualizar el servicio en ECS.

Por último, se abre la URL de Amplify y se comprueba que el frontend recibe datos del backend.

Después del deploy

Actualizaciones

- **Frontend:** un `git push` provoca un nuevo despliegue automático en Amplify.
- **Backend:** se construye y se sube la imagen nueva a ECR, y se actualiza el servicio en ECS.

Importante: limpiar al terminar

El balanceador de carga (ALB) y Fargate **cobran por hora** aunque nadie use la aplicación. Si se trata solo de un ejercicio, conviene **eliminar el servicio de ECS** y la app de Amplify al terminar, para evitar cargos inesperados.

Problemas frecuentes

Problema	Dónde revisar
No arranca el contenedor	Logs en CloudWatch y el task execution role
El health check falla	Que `/health` exista y que el puerto sea el correcto

Problema	Dónde revisar
El frontend no recibe datos	Consola del navegador (F12): casi siempre es CORS
404 al recargar en Angular	Regla de rewrite del Paso 8